

Задание 0. Проанализируйте содержимое файлов *Employee.cs*, *Worker.cs*, *Programmer.cs*, *Manager.cs*, *Program.cs*, а также вопросы по коду, которые могут быть заданы при защите задания 1 лабораторной и ответы на них

Вопросы, к содержимому и ответы на них

1. Вопрос: Какой класс является базовым (родительским) для всех остальных классов в этой системе?

Ответ: Employee

2. Вопрос: Перечислите все производные (дочерние) классы, которые наследуются от базового класса.

Ответ: Worker, Programmer, Manager

3. Вопрос: Какие свойства и методы наследуются производными классами от базового класса без изменений?

Ответ:

- Свойства: ID, Name, DateOfBirth, DateOfHiring, Age, Experience
- Методы: TotalSalary()
- Статическое свойство: StartSalary

4. Вопрос: В каких производных классах переопределен метод SalaryBonus() и как это влияет на полиморфное поведение?

Ответ: Метод SalaryBonus() переопределен во ВСЕХ производных классах.

Полиморфное поведение проявляется в том, что при вызове метода для объекта базового типа выполняется реализация производного класса.

5. Вопрос: Какие конструкторы используются в производных классах для вызова конструктора базового класса?

Ответ: Все производные классы используют ключевое слово `base()` в своих конструкторах для вызова конструктора базового класса.

6. Вопрос: Какой метод переопределен во всех производных классах для предоставления специфической информации о каждом типе сотрудника?

Ответ: Метод ToString()

7. Вопрос: В классе Programmer какое уникальное свойство добавляется при наследовании, которого нет в базовом классе?

Ответ: Свойство ProgrammerLevel типа перечисления Level

8. Вопрос: Как наследование позволяет хранить объекты разных типов (Worker, Programmer, Manager) в одном массиве Employee[]?

Ответ: Наследование позволяет это, так как все производные классы являются также и Employee (отношение "is-a").

9. Вопрос: Какие методы не переопределяются в производных классах и используются напрямую из базового класса?

Ответ: Метод TotalSalary() не переопределяется и используется напрямую из базового класса.

10. Вопрос: Как механизм наследования обеспечивает повторное использование кода в данной системе классов?

Ответ: Наследование обеспечивает повторное использование кода через наследование общих свойств и методов от базового класса, что исключает дублирование кода.

11. Вопрос: В методе TotalSalary() как работает вызов виртуального метода SalaryBonus() через наследование?

Ответ: В методе TotalSalary() вызывается виртуальный метод SalaryBonus(), и в runtime определяется фактический тип объекта, вызывается соответствующая реализация производного класса.

12. Вопрос: Какие модификаторы доступа используются в базовом классе для обеспечения правильного наследования?

Ответ: Используются модификаторы: public для открытого API, private для внутренней логики, virtual для методов, которые могут переопределяться.

13. Вопрос: Как статическое свойство базового класса StartSalary доступно в производных классах?

Ответ: Статическое свойство `StartSalary` доступно в производных классах через имя базового класса `Employee.StartSalary` или напрямую `StartSalary`.

14. Вопрос: Какие преимущества дает переопределение метода `ToString()` в каждом производном классе?

Ответ: Преимущества: единообразное представление объектов всех типов, легкая отладка и логирование, добавление специфической информации каждого производного класса.

15. Вопрос: Как полиморфизм проявляется при вызове методов `SalaryBonus()` и `TotalSalary()` для объектов разных типов, хранящихся в массиве `Employee[]`?

Ответ: Полиморфизм проявляется в том, что при обходе массива `Employee[]` вызов `emp.TotalSalary()` для каждого элемента приводит к выполнению своей логики расчета бонуса для каждого типа сотрудника, в зависимости от фактического типа объекта.